

Alto Mar — Arquitetura do chat banhista ↔ locador

Documento de referência para implementação do chat por reserva.
Versão: 1.0 · Junho 2026

Sumário

- 1. [Regras de produto](#)
- 2. [Visão geral do sistema](#)
- 3. [Modelo de dados](#)
- 4. [Módulo backend](#)
- 5. [API REST](#)
- 6. [Notificações](#)
- 7. [Frontend](#)
- 8. [Segurança e conformidade](#)
- 9. [Impacto no app](#)
- 10. [Ordem de implementação](#)
- 11. [Roadmap fase 2](#)

1. Regras de produto

Decisões fechadas para o MVP:

Regra	Comportamento
Quando abre	Só com reserva ACCEPTED (confirmada pelo locador)
Conteúdo	Texto plano; sem anexos
Após fim	COMPLETED ou CANCELLED → thread só leitura (histórico preservado)
Nunca abre	PENDING, DECLINED (nunca chegou a confirmada)
Anti-desintermediação	Rejeitar telefones, e-mails, URLs, @handles, palavras-chave (whatsapp, pix, etc.)
Notificação	Cada mensagem nova → notificação in-app + push FCM para a contraparte

Estados da reserva e permissões de chat

PENDING → chat bloqueado (sem thread)
DECLINED → chat bloqueado (nunca confirmada)
ACCEPTED → leitura + escrita
COMPLETED → só leitura (histórico)
CANCELLED → só leitura se existir histórico de quando estava ACCEPTED

Fluxo de estados (reserva)

```
[criada] → PENDING —aceite→ ACCEPTED —conclusão→ COMPLETED
          |
          | recusa
          ↓
        DECLINED
          |
          | cancelamento → CANCELLED
```

2. Visão geral do sistema

Modelo **assíncrono REST + polling** — encaixa no stack actual (Express + PostgreSQL no Railway, front na Vercel, app Android via Capacitor). Reutiliza o sistema de notificações existente (notifyUser + FCM). **Sem WebSocket** no MVP.

Componentes



Princípios arquiteturais

- **Uma thread por reserva** — chave natural: `booking_id`.
- **Participantes fixos** — apenas `renter_user_id` e `owner_user_id` da reserva.
- **Mensagens imutáveis** — sem edição nem apagar (auditoria e disputas).
- **Fonte de verdade no servidor** — validação de estado, conteúdo e rate limit sempre server-side.

3. Modelo de dados

3.1 Tabela `booking_messages`

Ficheiro de migração sugerido: `db/booking_messages.sql`

```
CREATE TABLE IF NOT EXISTS booking_messages (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  booking_id  uuid NOT NULL REFERENCES bookings(id) ON DELETE CASCADE,
  sender_user_id uuid NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
  body        text NOT NULL CHECK (char_length(trim(body)) BETWEEN 1 AND 2000),
  created_at  timestampz NOT NULL DEFAULT now()
);

CREATE INDEX idx_booking_messages_booking_created
  ON booking_messages (booking_id, created_at ASC);

CREATE INDEX idx_booking_messages_booking_id_desc
  ON booking_messages (booking_id, created_at DESC);
```

Campo	Descrição
booking_id	Reserva associada (thread única)
sender_user_id	Banhista ou locador
body	Texto plano, 1–2000 caracteres
created_at	Ordenação cronológica

3.2 Tabela `booking_message_reads`

Cursor de leitura por utilizador e reserva:

```
CREATE TABLE IF NOT EXISTS booking_message_reads (
  booking_id  uuid NOT NULL REFERENCES bookings(id) ON DELETE CASCADE,
  user_id     uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  last_read_at timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (booking_id, user_id)
);
```

Mensagens não lidas: `created_at > last_read_at AND sender_user_id ≠ user_id`.

3.3 Campo opcional em `bookings` (cache)

```
ALTER TABLE bookings ADD COLUMN IF NOT EXISTS last_message_at timestampz NULL;
```

Actualizado no `INSERT` de mensagem — acelera listagens “reservas com actividade recente”. Opcional no MVP.

4. Módulo backend

4.1 Estrutura de ficheiros

```
server/chat/
  schema.js          # ensureBookingChatSchema()
  access.js          # canAccessChat, canSendMessage, loadBookingParticipant
  contentFilter.js    # validateMessageBody(body) → { ok, reason? }
  messages.js        # list, send, markRead, unreadCounts
  routes.js          # rotas Express
  events.js          # notifyBookingMessageSent()
```

Registo em `server/index.js`: montar rotas + `ensureBookingChatSchema()` no arranque (padrão de `server/notifications/schema.js`).

4.2 Autorização (`access.js`)

```
// Regras canónicas (pseudocódigo)

function loadParticipantBooking(bookingId, userId) {
  // SELECT bk.* WHERE id = $1
  //   AND (renter_user_id = $2 OR owner_user_id = $2)
}

function canAccessChat(booking) {
  if (booking.status === 'ACCEPTED') return { mode: 'read_write' };
  if (booking.status === 'COMPLETED' || booking.status === 'CANCELLED') {
    return { mode: 'read_only' };
  }
  return { mode: 'hidden' }; // PENDING, DECLINED
}

function canSendMessage(booking) {
  return booking.status === 'ACCEPTED';
}
```

- Endpoints verificam **participação na reserva** (não basta JWT genérico).
- Endpoint unificado `/api/bookings/:id/chat/...` — evita duplicar rotas `owner/renter`.

4.3 Filtro de conteúdo (`contentFilter.js`)

Validação **antes** do `INSERT`. Resposta HTTP 400 com código para i18n.

Padrão	Exemplos bloqueados
Telefone BR	(11) 99999-9999, +55 21 ..., 10–11 dígitos seguidos
E-mail	foo@bar.com
URL	http://, https://, www., domínios .com, .br, etc.
Redes / apps	whatsapp, wa.me, telegram, instagram, @usuario
Pagamento externo	pix, chave pix, transferência, deposita

Implementação:

1. Normalizar: lowercase, colapsar espaços/pontuação em sequências numéricas.
2. Regex + lista negra configurável (CHAT_BLOCKED_KEYWORDS).
3. Mensagem ao utilizador: *"Não é permitido partilhar contactos ou links. Use o chat apenas para combinar detalhes do passeio."*
4. Log interno (sem PII exposta): booking_id, sender_user_id, blocked_reason.

4.4 Rate limit

Limite	Valor
Mensagens por hora	30 por (user_id, booking_id)
Intervalo mínimo	2 segundos entre envios consecutivos
Resposta	HTTP 429, código CHAT_RATE_LIMIT

5. API REST

Base: /api/bookings/:bookingId/chat/... — requer JWT; utilizador deve ser participante da reserva.

Método	Rota	Descrição
GET	/api/bookings/:id/chat/meta	{ mode, unreadCount, lastMessageAt, canSend }
GET	/api/bookings/:id/chat/messages? since=&limit=	Lista paginada (default 50, max 100)
POST	/api/bookings/:id/chat/messages	Corpo: { body: string }
POST	/api/bookings/:id/chat/read	Marca lidas até now()
GET	/api/chat/unread-summary	{ totalUnread, byBooking: [{ bookingId, count }] }

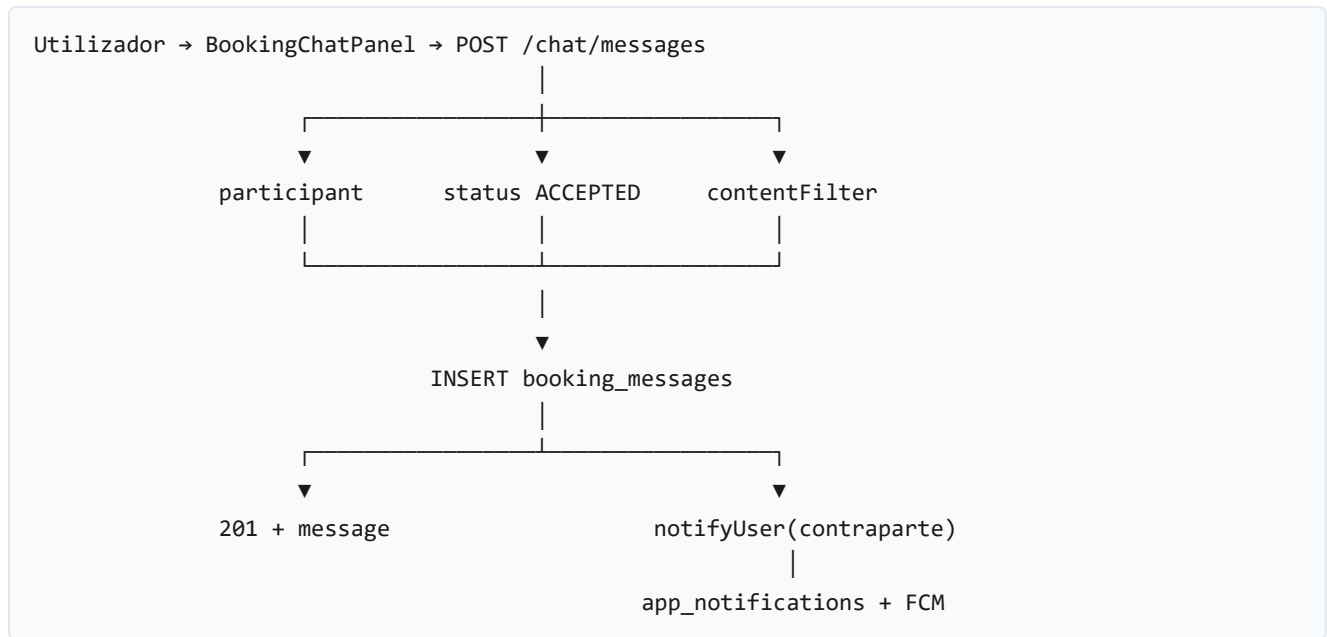
5.1 Exemplo — GET messages

```
{
  "mode": "read_write",
  "messages": [
    {
      "id": "uuid",
      "senderUserId": "uuid",
      "senderRole": "banhista",
      "body": "Qual o ponto de embarque exacto?",
      "createdAt": "2026-06-07T14:30:00.000Z"
    }
  ],
  "hasMore": false
}
```

5.2 Códigos de erro

HTTP	Código	Situação
403	CHAT_NOT_AVAILABLE	PENDING / DECLINED
403	CHAT_READ_ONLY	POST em COMPLETED / CANCELLED
400	CHAT_CONTENT_BLOCKED	telefone, link ou palavra proibida
429	CHAT_RATE_LIMIT	demasiadas mensagens

5.3 Fluxo de envio



6. Notificações

Integração com server/notifications/service.js e bookingEvents.js.

6.1 Novo tipo

```
export const NotificationType = {  
  // ... existentes  
  BOOKING_MESSAGE: "BOOKING_MESSAGE",  
};
```


6.2 Evento por mensagem

```
export async function notifyBookingMessage({ bookingId, senderUserId, previewBody }) {
  const ctx = await loadBookingContext(bookingId);
  const recipientId =
    senderUserId === ctx.renter_user_id ? ctx.owner_user_id : ctx.renter_user_id;
  const isOwnerRecipient = recipientId === ctx.owner_user_id;

  const path = isOwnerRecipient
    ? `/marinheiro/reservas/${bookingId}#chat`
    : `/conta/reservas#booking-${bookingId}-chat`;

  await notifyUser({
    userId: recipientId,
    type: NotificationType.BOOKING_MESSAGE,
    title: "Nova mensagem sobre a reserva",
    body: truncate(previewBody, 120),
    path,
    bookingId,
    data: { senderUserId, bookingId },
    sendPush: true,
  });
}
```

6.3 Leitura cruzada

- Ao abrir ficha com #chat: POST /api/bookings/:id/chat/read.
- Extender markNotificationsReadForVisit para marcar notificações BOOKING_MESSAGE do mesmo booking_id.
- Push FCM: deep link para a secção de chat na ficha da reserva.

7. Frontend

7.1 Componentes

src/components/chat/	
BookingChatPanel.tsx	# thread + estados
BookingChatMessage.tsx	# bolha (eu / outro)
BookingChatInput.tsx	# textarea + enviar
useBookingChat.ts	# fetch, poll, send, markRead
chatApi.ts	# chamadas REST

7.2 Superfícies de integração

Papel	Ficheiro	Quando mostrar
Banhista	RenterBookingsPanel.tsx	ACCEPTED, COMPLETED, CANCELLED (com histórico)
Locador	OwnerBookingDetailPage.tsx	Idem — secção "Mensagens"

mode	UI
hidden	Secção não renderizada
read_write	Thread + input activo
read_only	Thread + banner "Conversa encerrada"; sem input

7.3 Polling (sem WebSocket)

Contexto	Intervalo
Chat aberto e tab activa	5 s (GET ?since=lastCreatedAt)
Lista de reservas	60 s (GET /chat/unread-summary)
App em background (Android)	Parar polling; FCM traz utilizador de volta

7.4 UX mínima

- Ordem cronológica ascendente; auto-scroll ao enviar/receber.
- Identificação: "Tu" vs nome da contraparte / barco.
- Erro de conteúdo bloqueado: toast; manter texto no input.
- Badge de não lidas no card da reserva.
- i18n: chaves `bookingChat.*` (pt, en, es).

8. Segurança e conformidade

Tema	Abordagem
Autorização	Só participantes; IDs validados server-side
Auditoria	Mensagens imutáveis
LGPD	Retenção alinhada à reserva; cascade em delete
Moderação	Filtro automático; export admin futuro
XSS	Texto plano escapado; sem HTML na renderização

9. Impacto no app

Área	Alteração
Base de dados	2 tabelas novas + migration
server/index.js	Rotas chat + schema no boot
Notificações	+1 tipo, +1 evento, ajuste mark-read
Badges	Incluir unread de chat no sino ou badge na ficha
docs/BUSINESS-RULES.md	Nova secção "Chat por reserva"
Android / Capacitor	Sem plugins novos; FCM existente
Stripe / pagamentos	Nenhum impacto

Estimativa MVP: ~12–18 ficheiros · ~1–1,5 semanas.

10. Ordem de implementação

```
1. Schema SQL + access.js
  ↓
2. contentFilter.js + POST messages
  ↓
3. GET messages + read cursor + unread summary
  ↓
4. notifyBookingMessage (in-app + FCM)
  ↓
5. BookingChatPanel + hook
  ↓
6. Integrar OwnerBookingDetailPage + RenterBookingsPanel
  ↓
7. Badges, mark-read cruzado, i18n
  ↓
8. Testes + atualizar BUSINESS-RULES.md
```

Casos de teste essenciais

- Enviar com reserva ACCEPTED → 201.
- PENDING → 403 CHAT_NOT_AVAILABLE.
- COMPLETED → GET OK; POST → 403 CHAT_READ_ONLY.
- Corpo com telefone/URL → 400 CHAT_CONTENT_BLOCKED.
- Contraparte recebe app_notifications + push.
- Unread decrementa após POST /read.
- Cancelamento: histórico permanece legível.

11. Roadmap fase 2

Fora do MVP acordado:

- Typing indicator / WebSocket ou serviço realtime (Ably, etc.)
- Denunciar mensagem
- Painel admin de moderação
- Agrupamento de push ("3 mensagens novas")
- Tradução automática

Referências no repositório

Recurso	Localização
Notificações	server/notifications/service.js
Eventos de reserva	server/notifications/bookingEvents.js
Schema reservas	db/schema.sql → bookings
Ficha locador	src/pages/owner/OwnerBookingDetailPage.tsx
Reservas banhista	src/components/RenterBookingsPanel.tsx
Regras de negócio	docs/BUSINESS-RULES.md (secção 13 — notificações)
Push Android	docs/ANDROID.md

Alto Mar · Documento interno de arquitectura